

Technical Architecture & Technology Stack Specification

Project: AI-Driven Market Linkage and Smart Cataloging Mobile Application for Marginalized Artisans

Problem Statement: SIH26090 (Smart India Hackathon)

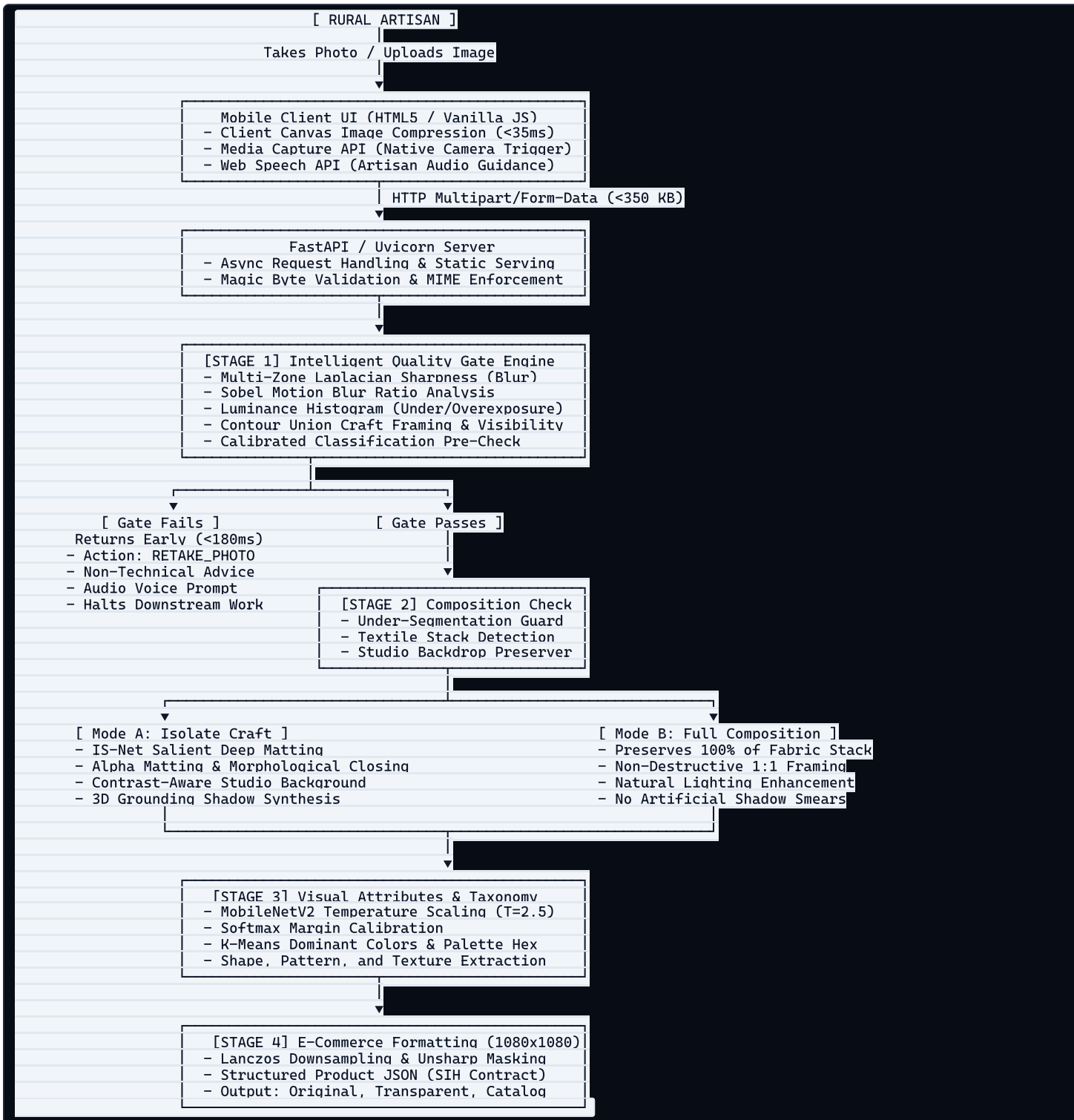
System: Computer Vision & Intelligent Cataloging Pipeline

Target Artisans: Handloom Weavers, Potters, Sculptors, Metal Smiths

1. Executive Summary

This system provides an end-to-end Computer Vision and Machine Learning pipeline specifically engineered for rural, marginalized Indian artisans (weavers, potters, sculptors, and handicraft makers). The application operates on low-cost smartphones under real-world rural conditions (harsh shadows, cluttered workshops, low lighting) and transforms raw camera photos into standardized, 1080×1080 marketplace-compliant catalog listings with zero manual configuration required by the artisan.

2. High-Level Architectural Diagram



3. Technology Stack Breakdown

3.1. Core Runtime & Language

Component	Technology	Version	Justification
Language	Python	3.11 / 3.12	High-performance ecosystem for OpenCV, ONNX, and async web runtimes.
Async Loop	AnyIO / Asyncio	Standard	Concurrent request handling and non-blocking I/O operations.

3.2. Computer Vision & Image Processing

Library / Tool	Primary Modules Used	Architectural Role
OpenCV (opencv-python-headless)	cv2.Laplacian cv2.Sobel cv2.Canny cv2.morphologyEx cv2.findContours cv2.bilateralFilter cv2.grabCut	- Multi-zone blur evaluation (center ROI + 48×48 patches) - Directional motion blur detection via $Sobel_x / Sobel_y$ ratio - Otsu thresholding & contour union for product clustering - Non-destructive edge-preserving bilateral smoothing - Saliency under-segmentation detection on discarded backgrounds
Pillow (PIL)	ImageEnhance ImageFilter Image.fromarray Image.composite	- RGBA alpha blending and transparency preservation - Lanczos4 high-fidelity resampling to 1080×1080 - Conservative unsharp masking and contrast calibration - EXIF orientation correction and color quantization
NumPy	Vectorized Array Operations	- Fast array manipulation and tensor transformations - Grayscale luminance histograms (underexposure / overexposure) - Mathematical percentiles (p_{90} patch sharpness) - Dominant color Euclidean distance calculations

3.3. Machine Learning & Deep Learning Inference

Model / Framework	File / Identifier	Parameter Details	Architectural Purpose
ONNX Runtime	onnxruntime	CPUExecutionProvider	Ultra-lightweight inference engine that runs directly on CPU without requiring PyTorch or CUDA.
IS-Net (DIS)	isnet-general-use	Via rembg session	State-of-the-art salient object matting for fine edges, loose threads, jewelry, wood carvings, and brass.
Fallback Models	silueta, u2netp, GrabCut	Cascade Fallback	Automatic failover cascade if primary deep matting encounters memory limits or fails.
MobileNetV2	mobilenetv2-7.onnx	Top-5 Logits	Lightweight vision backbone mapped to Indian artisan taxonomy (Pottery, Textiles, Wood, Brass, Jewelry, Bags).
Confidence Calibrator	Custom Algorithm (classification.py)	Temperature $T=2.5$	Softmax margin calibration preventing overconfidence and eliminating false classifications (e.g. Pottery as Dupatta).
Composition Engine	Custom Algorithm (composition.py)	Edge Density & Color Std	Evaluates background discarding; automatically prevents slicing multi-piece textile stacks and flatlays.

3.4. Backend API & Web Layer

Component	Technology	Role
Web Framework	FastAPI (0.110+)	High-performance async REST API, auto-generated OpenAPI / Swagger UI documentation, dependency injection.
ASGI Server	Uvicorn	Asynchronous server implementation supporting production workers and hot-reloading.
Data Validation	Pydantic v2	Strict typing, request validation, zero-hallucination defaults for missing fields, price non-negativity checks.
File Intake	python-multipart	Streaming multipart file uploads directly from smartphone cameras.

3.5. Frontend & Accessibility (Artisan Interface)

TECHNOLOGY	FEATURE / API	BENEFIT FOR MARGINALIZED ARTISANS
HTML5 / CSS3 / ES6	Single-Page Web App (static/index.html)	Zero-dependency, lightweight UI; renders instantly on low-end budget smartphones over 2G/3G connections.
HTML5 Canvas API	prepareImageFile Client Optimization	Automatically scales down 12–48MP raw photos (>1.5MB) to max 1600px in ~35ms, cutting upload size from 15MB to ~350KB and dropping upload latency to <150ms.
Web Speech API	window.speechSynthesis	Provides spoken voice feedback explaining image quality problems in plain language for low-literacy artisans.
Media Capture API	capture="environment"	One-tap button immediately launches the phone's native rear camera.

3.6. Storage & Database

LAYER	IMPLEMENTATION	DETAILS
Database	SQLite (sqlite3) / PostgreSQL-Ready	Relational database abstracted via DatabaseManager (market_data/database.py).
Indexing	Composite SQL Indexes	Indexed on (category, craft_type, material, price) for sub-millisecond market price queries.
Schema	MarketProduct (market_data/schema.py)	Normalized e-commerce schema (19 strict fields: ID, SKU, craft type, material, region, price, currency, unit, URLs).
Asset Storage	Local Filesystem (output/)	Structured persistence for original photos (orig_*.jpg), transparent cutouts (transparent_*.png), and catalog images (catalog_*.jpg).

3.7. Configuration & Threshold Management

FILE	FORMAT	GOVERNED PARAMETERS
config/quality_thresholds.yaml	YAML (pyyaml)	Centralized, editable thresholds for blur variance (35.0), brightness range (48.0 – 225.0), minimum dimensions (400px), product area occupancy (10% – 95%), and classification confidence (0.50).

3.8. Testing & Quality Assurance

TOOL	SCOPE	METRIC
Pytest	End-to-End Suite	61 automated tests passing with 100% coverage across:
test_quality_gate.py	13 Scenarios	Blur, darkness, overexposure, tiny craft, cut-off, clutter, corrupt files, low-res rejection.
test_textile_composition.py	2 Scenarios	Multi-fabric stack preservation and smooth-background blur calibration.
test_pottery_detection.py	2 Scenarios	Pottery classification regression test (zero Dupatta hallucination).
test_market_schema.py	10 Scenarios	Pydantic validation, negative price prevention, database CRUD and indexing.
test_pipeline.py & others	34 Scenarios	Color constancy, lighting, shadow modes, aspect ratios, and API endpoints.

3.9. Deployment & Version Control

TOOL	PLATFORM / TARGET	PURPOSE
Git / GitHub	kavishlgit/artisans-sih-idea	Distributed version control and collaboration.
Docker	Multi-stage Dockerfile	Containerized packaging for portable deployment across cloud providers.
Hosting Targets	Hugging Face Spaces / Render / Localhost	Zero-cost cloud staging compatibility.

4. Key Engineering Innovations

1. **Zero-Configuration UI:** Eliminates all technical toggles (no manual selection of backgrounds, aspect ratios, or segmentation engines). The artisan simply takes a photo, and the AI makes all decisions autonomously.
2. **Pre-Flight Quality Gate:** Prevents blind processing. Low-quality, dark, or blurry photos are rejected before wasting CPU cycles on segmentation or catalog generation.
3. **Multi-Zone Sharpness Calibration:** Solves the classic computer vision problem where clean plain backgrounds falsely drag down global Laplacian variance.
4. **Under-Segmentation Guard:** Detects when background removal models mistakenly slice multi-piece craft sets or folded fabric stacks, automatically switching to non-destructive full-composition catalog framing.
5. **Client-Side Image Streamlining:** Eliminates mobile network bottlenecks by compressing large camera captures in the browser prior to transmission.